॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

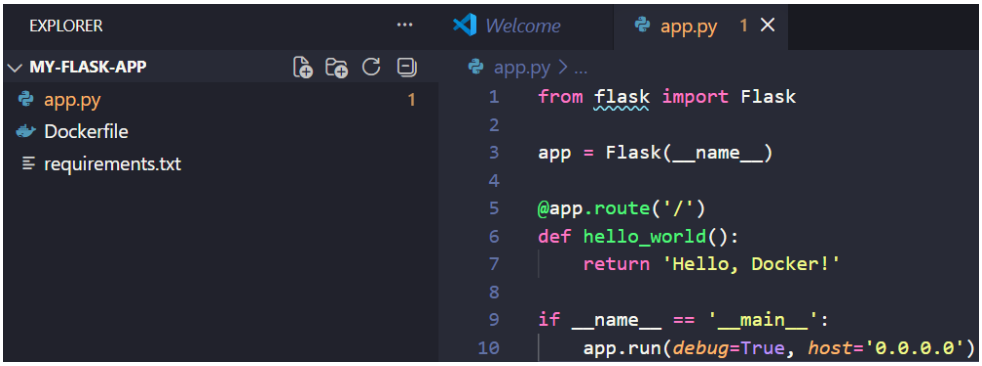
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 08

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	**Deepak Rathod**
PRN No.	**23070122175**
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2026, Sem VII
Date of Performance	23rd July, 2026
Title of Assignment:	Dockerfile & Image Build
Theory:	Docker is a containerization platform that enables applications and their dependencies to be packaged into lightweight, portable containers. A Dockerfile contains the instructions required to build a Docker image, while a Docker image serves as a template for creating containers. A Docker container is a running instance of an image that provides an isolated environment for executing applications consistently across different systems. In this experiment, a simple Flask web application was containerized by creating the required application files, building a Docker image, running the container, and verifying the application through a web browser. **Commands Used** • mkdir my-flask-app – Created a new directory for the Flask application. • cd my-flask-app – Navigated to the project directory. • docker build -t my-flask-app . – Built a Docker image named my-flask-app from the Dockerfile. • docker build --no-cache -t my-flask-app . – Rebuilt the Docker image without using cached layers after updating the dependencies. • docker run -p 5000:5000 my-flask-app – Started the Docker container and mapped port 5000 of the container to port 5000 on the host machine for browser access.

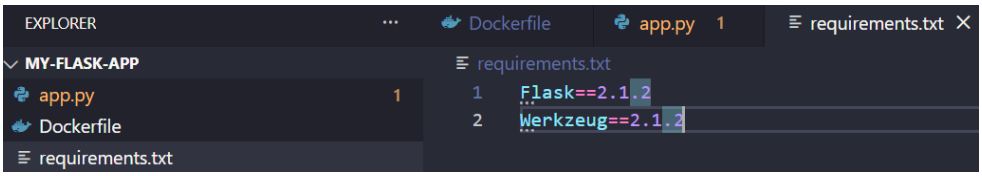
Implementation:

Step 1: Create the Flask Application



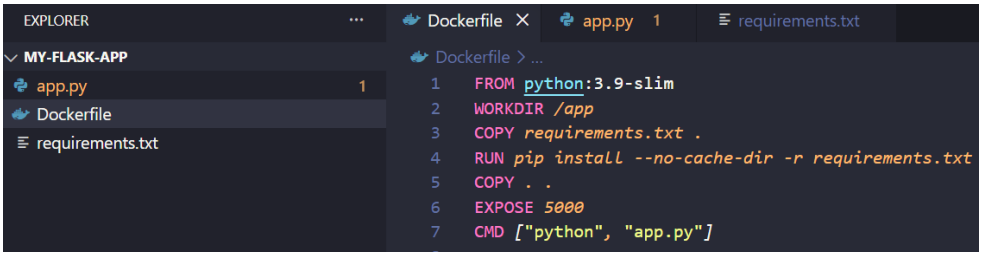
Created Flask Application — Created the my-flask-app project directory and added the required application files. Implemented the app.py file containing a simple Flask web application that displays the message "Hello, Docker!" when accessed.

Step 2: Create the Requirements File



Created Requirements File — Created the requirements.txt file specifying the Flask dependency required to install the necessary packages for the Flask application.

Step 3 – Create the Dockerfile



Created Dockerfile — Created the Dockerfile specifying the Python base image, working directory, application dependencies, exposed port, and startup command required to containerize the Flask application.

Step 4 – Build the Docker Image

```
PS C:\Users\riya1\OneDrive\Desktop\my-flask-app> docker build -t my-flask-app .
[+] Building 8.7s (10/10) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 201B
 => [internal] load metadata for docker.io/library/python:3.9-slim
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b
 => => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b
 => [internal] load build context
 => => transferring context: 115B
 => CACHED [2/5] WORKDIR /app
 => [3/5] COPY requirements.txt .
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => [5/5] COPY . .
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:fb89f60f574a9429e931ce33e9572dd7e1d86ccd7e9db9b8c4c15a67f5c713fd
 => => exporting config sha256:52c5774faf20cef4f21c9ea3373f3417a45259dae51f21fa566a394458d0e808
 => => exporting attestation manifest sha256:7743d1d7e3056c122e4af479163f8fefcfd1b5a3fb0f7f58bd0fe2f6e85d8b7a
 => => exporting manifest list sha256:a35864ab009b709ab04622586c1aa6dec727e76887dd39a7ccae3c3dc7e5ced0
 => => naming to docker.io/library/my-flask-app:latest
 => => unpacking to docker.io/library/my-flask-app:latest
```

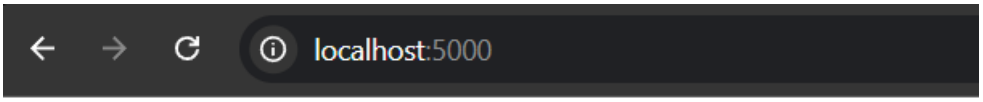
Built Docker Image — Successfully built the Docker image named my-flask-app using the Dockerfile. Docker completed all build stages and created the container image successfully.

Step 5 – Run the Docker Container

```
PS C:\Users\riya1\OneDrive\Desktop\my-flask-app> docker run -p 5000:5000 my-flask-app
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 908-927-277
172.17.0.1 - - [23/Jul/2026 04:10:43] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [23/Jul/2026 04:10:43] "GET /favicon.ico HTTP/1.1" 404 -
172.17.0.1 - - [23/Jul/2026 04:10:50] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [23/Jul/2026 04:11:03] "GET / HTTP/1.1" 200 -
```

Executed Docker Container — Successfully started the Docker container and launched the Flask application. The application is running on port 5000 and is accessible through the host machine.

Step 6: Verify the Flask Application



Hello, Docker!

Verified Dockerized Flask Application — Opened the Flask application in a web browser using http://localhost:5000 and successfully verified that the Docker container served the "Hello, Docker!" webpage.

Subject Faculty Name: Prof. Arijit Dutta